Adaptive Signal Processing

Assignment

Submitted by : Roopesh O R

1 Random Search

Random search (RS) is a family of numerical optimization methods that do not require the gradient of the optimization problem, and RS can hence be used on functions that are not continuous or differentiable. They're also known as direct-search/derivative-free/black-box methods. Mainly there are 2 types:

- Lienar random search
- Genetic Optimizer

1.1 Linear Random Search (LRS)

Here a small random change – random vector U_k (with covariance $= \sigma^2 I$) is added to weight vector at the beginning of each iteration. that is:

$$W_{k+1} = W_k + \frac{\mu}{\sigma^2}\left[\hat{\xi}(W_k) - \hat{\xi}(W_k + U_k)\right]U_k$$

where

- W_k = weight vector
- μ, σ = convergance parameter
- $\hat{\xi}(W_k), \hat{\xi}(W_k + U_k)$ = estimated MSE of present and tentative weight vectors.

The behaviour of LRS is similar to that of steepest descent. As shown in table below, the effect of μ has same effect on misadjustment and time constant in both algorithms. LRS is simpler to implement compared to steepest descent. But it is less efficent than LMS

	LRS	**Steepest Descent**
Misadjustment	$\frac{\mu(L+1)}{N\sigma^2}\xi_{\min} = \frac{(L+1)^2}{4P}\left(\frac{1}{T_{\text{mse}}}\right)_{\text{av}}$	$\frac{\mu(L+1)}{N\delta^2}\xi_{\min} = \frac{(L+1)^2}{8P}\left(\frac{1}{T_{\text{mse}}}\right)_{\text{av}}$
Perturbation, P	$\frac{\sigma^2(L+1)\lambda_{\text{av}}}{2\xi_{\min}}$	$\frac{\delta^2\lambda_{\text{av}}}{\xi_{\min}}$
Time constant of nth mode	$\frac{1}{4\mu\lambda_n}$	$\frac{1}{4\mu\lambda_n}$

1.2 Genetic Optimizer

This is a genetic algorithm. Steps:

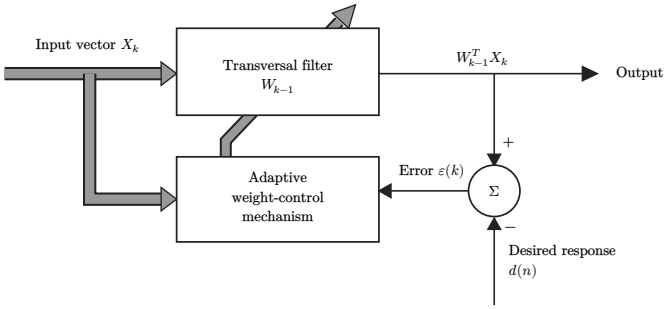
1. a population of M values of weight w are created at random
2. $\xi(w)$ is estimated for each of these values
3. Revised population of $N > M$ values are created. Here M values are replicated in inverse proportion to ξ (wights with lower ξ are replicated more).
4. Next, in the 2nd generation, offspring are created as follows. Two values of w are chosen at random and, at an arbitrary point, the binary representations of the two values are split. The first segment of one binary string is concatenated with the second segment of the other to yield the offspring. Eg:

$$a = a_1 a_2 a_3 a_4 a_5 a_6 a_7 a_8$$
$$b = b_1\ b_2 b_3\ b_4 b_5\ b_6 b_7\ b_8$$
$$\text{(offspring)} \quad c = a_1 a_2 a_3 a_4 b_5\ b_6 b_7\ b_8$$

The 2nd generation is formed by creating M offspring in this manner and replacing all of the parents in the first generation. The population of the second generation is again increased from M to N as above, replicating values of w that have lower values of $\xi(w)$, and so on. The entire process converges as the offspring become more and more alike, and may be made to diverge or search the error space by introducing random mutations.

2 RLS Algorithm (Recursive Least Squares)

RLS algorithm is an adaptive filtering algorithm used to estimate the optimum filter coefficients by minimizing the sum of squared errors. Unlike the LMS, RLS uses information from previous observations to estimate the optimum coefficients. It is particularly useful when fast convergence is required and the input signal has highly correlated samples.



2.1 Formulation

The goal is to minimize:

$$\xi = \sum_{i=0}^{k} e_i^2$$

where

$$e_k = d_k - W^T x_k$$

Minimizing ξ and solving gives:

$$W_k = R_k^{-1} P_k$$

$$R_k = \sum_{i=0}^{k} X_i X_i^T \qquad \text{and} \qquad P_k = \sum_{i=0}^{k} X_i d_i$$

To make this recursive consider next iteration:

$$R_{k+1} = \sum_{i=0}^{k+1} X_i X_i^T$$
$$\Rightarrow R_{k+1} = R_k + X_{k+1} X_{k+1}^T$$

Similarly:

$$P_{k+1} = P_k + d_{k+1} X_{k+1}$$

The weight vector is

$$W_{k+1} = R_{k+1}^{-1} P_{k+1}$$

To calculate R_{k+1}^{-1} we use *matrix inversion lemma*:

Let $Q_k = R_k^{-1}$, RLS gain vector is:

$$K_k = \frac{Q_{k-1} X_k}{1 + X_k^T Q_{k-1} X_k}$$

The a-priori error is:

$$e_k = d_k - W_{k-1}^T X_k$$

updated wight update equation becomes:

$$W_k = W_{k-1} + K_k e_k$$

The inverse correlation matrix is updated as:

$$Q_k = Q_{k-1} - \frac{Q_{k-1} X_k X_k^T Q_{k-1}}{1 + X_k^T Q_{k-1} X_k}$$

2.1.1 With forgetting factor:

In practical adaptive filtering applications, the input statistics may change with time. If all previous observations are given equal importance, the algorithm may become slow in responding to changes in the system. To overcome this, a forgetting factor λ is introduced. The modified cost function becomes exponentially weighted sum of squared error:

$$\xi_k = \sum_{i=0}^{k} \lambda^{k-1} e_i^2 \qquad\qquad 0 < \lambda \leq 1$$

Where $\lambda < 1 \Rightarrow$ older observations gradually lose importance. The gain vector now is:

$$K_k = \frac{Q_{k-1} X_k}{\lambda + X_k^T Q_{k-1} X_k}$$

weight update equation becomes remains smae. The inverse correlation matrix is updated as:

$$Q_k = \frac{1}{\lambda}[Q_{k-1} - \left(K_k X_k^T Q_{k-1}\right]$$

3 Matrix Inversion Lemma

This is a mathematical identity that allows the inverse of a modified matrix to be calculated using the inverse of an existing matrix. It is also known as the Woodbury identity. This allows efficient computation of matrix inverses and solutions to linear systems after low-rank modifications. This identity is particularly important in algorithms such as RLS which require the inverse of an input correlation matrix.

The general matrix inversion lemma is

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U\left(C^{-1} + VA^{-1}U\right)^{-1}VA^{-1}$$

where

- A is an invertible matrix.

- U and V are appropriately dimensioned matrices.
- C is an invertible matrix.

if U, V are vectors, it reduces to:

$$(A + UV^T)^{-1} = A^{-1} - \left(\frac{A^{-1}UV^TA^{-1}}{1 + V^TA^{-1}U}\right)$$

Here UV^T is the outer product of vectors U, V

4 Convergence

Convergence refers to the process by which the filter coefficients approach their optimum values as more input-output samples are processed. In an adaptive filter, the coefficients are continuously modified according to the estimation error. The objective is to reach a state where the error becomes sufficiently small and the coefficient values become close to their optimum values. Consider an adaptive filter with output

$$y_k = w_k^T x_k$$

The error is

$$e_k = d_k - y_k$$

The objective is to minimize the mean-square error:

$$\xi(w) = E[e_k^2]$$

From this optimum coefficient vector is

$$W = R^{-1}P$$

Convergence means that the adaptive coefficient vector approaches this optimum solution.

4.1 Convergence of RLS

RLS minimizes the exponentially weighted SSE. Initially, the coefficients may be far from their optimum values. Consequently, the estimation error can be relatively large. As more samples arrive:

1. The algorithm gathers more information about the input signal.
2. The inverse correlation matrix is updated.
3. The gain vector determines the required coefficient correction.
4. The coefficient vector moves towards the least squares solution.
5. The estimation error generally decreases as the estimate improves.

Under suitable conditions, the algorithm approaches the optimum least squares coefficient vector.

4.2 Convergence speed

Convergence speed indicates how quickly an adaptive algorithm approaches its optimum coefficient values. RLS generally converges faster than LMS. The reason is that RLS uses the inverse input correlation matrix to adjust its coefficient updates. LMS, on the other hand, uses a gradient-based update with a fixed step-size parameter.

LMS convergence is strongly affected by the eigenvalue spread of the input correlation matrix. The eigenvalue spread is

$$\kappa = \frac{\lambda_{\max}(R)}{\lambda_{\min}(R)}$$

where $\lambda_{\max}(R)$ and $\lambda_{\min}(R)$ are the largest and smallest eigenvalues. A large eigenvalue spread generally causes slower convergence of LMS. RLS compensates for differences in input signal scaling and correlation through its inverse correlation matrix.

4.3 Factors affecting convergence of RLS

Forgetting factor (λ)

The forgetting factor controls how much importance is given to previous observations.

- A value close to 1 produces a longer memory.
- A smaller value gives more importance to recent observations.

Input signal characteristics

The statistical properties of the input signal influence the information available for estimating the filter coefficients. For reliable estimation, the input should sufficiently excite the filter. This property is commonly described using the term *persistent excitation.*

Filter order

The number of filter coefficients affects computational requirements and estimation complexity. A higher filter order generally requires more data and computational resources.

Numerical precision

RLS involves repeated matrix updates. Finite-precision arithmetic can introduce round-off errors, which may affect the symmetry and positive definiteness of the inverse correlation matrix.